

General Resource and Effect Grades

Mariana Milicich

Joint work with Danel Ahman

University of Tartu

May 26, 2026

Foundations Seminar

Contents of this Talk

- 1 Motivating Example
- 2 Modal Logic and Modal Types
- 3 The $\lambda_{[\tau]}$ -Calculus
- 4 Generalising $\lambda_{[\tau]}$ – Types
- 5 Generalising $\lambda_{[\tau]}$ – Operational Semantics
- 6 Conclusions and Ongoing/Future Work

1 Motivating Example

2 Modal Logic and Modal Types

3 The $\lambda_{[\tau]}$ -Calculus

4 Generalising $\lambda_{[\tau]}$ – Types

5 Generalising $\lambda_{[\tau]}$ – Operational Semantics

6 Conclusions and Ongoing/Future Work

Robotic Arms

```
let (partA , partB) = disassemble product in
```

```
let partB' = clean partB in
```

```
let partB'' = primer partB' in
```

```
let partB''' = paint partB'' in
```

```
let product' = assemble (partA , partB''') in
```

```
...
```



Robotic Arms

```
let (partA , partB) = disassemble product in  
# partA and partB available to use below  
let partB' = clean partB in  
  
let partB'' = primer partB' in  
  
let partB''' = paint partB'' in  
  
let product' = assemble (partA , partB''') in  
  
...
```



Robotic Arms

```
let (partA , partB) = disassemble product in  
# partA and partB available to use below  
let partB' = clean partB in  
# cleaning liquid has to evaporate before using partB' below  
let partB'' = primer partB' in  
  
let partB''' = paint partB'' in  
  
let product' = assemble (partA , partB''') in  
  
...
```



Robotic Arms

```
let (partA , partB) = disassemble product in  
# partA and partB available to use below  
let partB' = clean partB in  
# cleaning liquid has to evaporate before using partB' below  
let partB'' = primer partB' in  
# primer has to dry before using partB'' below  
let partB''' = paint partB'' in  
  
let product' = assemble (partA , partB''') in  
  
...
```



Robotic Arms

```
let (partA , partB) = disassemble product in
# partA and partB available to use below
let partB' = clean partB in
# cleaning liquid has to evaporate before using partB' below
let partB'' = primer partB' in
# primer has to dry before using partB'' below
let partB''' = paint partB'' in
# paint has to dry before using partB''' below
let product' = assemble (partA , partB''') in

...
```



Robotic Arms

```
let (partA , partB) = disassemble product in
# partA and partB available to use below
let partB' = clean partB in
# cleaning liquid has to evaporate before using partB' below
let partB'' = primer partB' in
# primer has to dry before using partB'' below
let partB''' = paint partB'' in
# paint has to dry before using partB''' below
let product' = assemble (partA , partB''') in
# product' available to use below
...
```



Robotic Arms

```
let (partA , partB) = disassemble product in
# partA and partB available to use below
let partB' = clean partB in
# cleaning liquid has to evaporate before using partB' below
let partB'' = primer partB' in
# primer has to dry before using partB'' below
let partB''' = paint partB'' in
# paint has to dry before using partB''' below
let product' = assemble (partA , partB''') in
# product' available to use below
...
```



graded modal types + graded effect system

- 1 Motivating Example
- 2 Modal Logic and Modal Types
- 3 The $\lambda_{[\tau]}$ -Calculus
- 4 Generalising $\lambda_{[\tau]}$ – Types
- 5 Generalising $\lambda_{[\tau]}$ – Operational Semantics
- 6 Conclusions and Ongoing/Future Work

Modal Logic and Modal Types

We can use modal types to express the different program modes.

Modal operators you may have seen before:

- $\Box X$

- $\Diamond X$

Modal Logic and Modal Types

We can use modal types to express the different program modes.

Modal operators you may have seen before:

- $\Box X$
- $\Diamond X$

Semantics of Modal Logic - Kripke's Possible World Semantics

- Fix a Kripke frame (set of worlds W and accessibility relation $\mathcal{R}$).
- The relation $\mathcal{R}$ may have different additional properties.
- The modalities operators are given interpretation with respect to W and $\mathcal{R}$:
 - $w \models \Box X \iff \forall w'. w' \mathcal{R} w \implies w' \models X$
 - $w \models \Diamond X \iff \exists w'. w' \mathcal{R} w \wedge w' \models X$

We are interested in proof systems for modal logic based on **natural deduction**:

- Formulas $\iff$ grammar of types
- Natural deduction rules $\iff$ typing rules
- Proof normalisation $\iff$ reduction rules

We are interested in proof systems for modal logic based on **natural deduction**:

- Formulas $\iff$ grammar of types
- Natural deduction rules $\iff$ typing rules
- Proof normalisation $\iff$ reduction rules

Problem: natural deduction for modal logic is nontrivial.

Providing an introduction rule for the $\Box$ operator is the main challenge.

One approach to a correct modal logic system is the one by Clouston.

- A modality $\boxplus$ acting on contexts:

$$\Gamma ::= \emptyset \mid \Gamma, X \mid \Gamma, \boxplus$$

- The $\boxplus$ modality internalises Kripke semantics abstractly:

$$\frac{\boxplus \notin \Gamma'}{\Gamma, X, \Gamma' \vdash X}$$

$$\frac{\Gamma, \boxplus \vdash X}{\Gamma \vdash \Box X}$$

$$\frac{\Gamma \vdash \Box X \quad \boxplus \notin \Gamma'}{\Gamma, \boxplus, \Gamma' \vdash X}$$

Fitch-Style Modal Types

- A modality $\mathbf{\Box}$ acting on (typing) contexts:

$$\Gamma ::= \emptyset \mid \Gamma, x : X \mid \Gamma, \mathbf{\Box}$$

- The $\mathbf{\Box}$ modality internalises Kripke semantics abstractly:

$$\frac{\mathbf{\Box} \notin \Gamma'}{\Gamma, x : X, \Gamma' \vdash x : X} \qquad \frac{\Gamma, \mathbf{\Box} \vdash \text{exp} : X}{\Gamma \vdash \mathbf{box} \text{ exp} : \Box X}$$

$$\frac{\Gamma \vdash \text{exp} : \Box X \quad \mathbf{\Box} \notin \Gamma'}{\Gamma, \mathbf{\Box}, \Gamma' \vdash \mathbf{unbox} \text{ exp} : X}$$

- 1 Motivating Example
- 2 Modal Logic and Modal Types
- 3 The $\lambda_{[\tau]}$ -Calculus
- 4 Generalising $\lambda_{[\tau]}$ – Types
- 5 Generalising $\lambda_{[\tau]}$ – Operational Semantics
- 6 Conclusions and Ongoing/Future Work

Values, computations, and operation clauses:

$$V, W ::= x \mid f(V_1, \dots, V_n) \mid () \mid (V, W) \mid \text{fun}(x : X) \mapsto M$$

$$\begin{aligned} M, N ::= & \text{return } V \mid \text{let } x = M \text{ in } N \mid V \ W \mid \text{match } V \text{ with } \{(x, y) \mapsto y\} \\ & \mid \text{op } V \ (x.M) \mid \text{delay } \tau \ M \mid \text{handle } M \text{ with } H \text{ to } x \text{ in } N \\ & \mid \text{box}_{[\tau]X} V \text{ as } x \text{ in } M \mid \text{unbox}_{[\tau]X} V \text{ as } x \text{ in } M \end{aligned}$$

$$H ::= (x.k.M_{\text{op}})_{\text{op} \in \mathcal{O}}$$

Let $\tau \in \mathbb{N}$.

Types and typing contexts:

$$A, B ::= \mathbf{b} \mid \mathbf{unit} \mid A \times B \mid [\tau]A$$

$$X, Y ::= A \mid X \times Y \mid X \rightarrow Y!_{\tau} \mid [\tau]X$$

$$C ::= X!_{\tau}$$

$$\Gamma ::= \emptyset \mid \Gamma, x : X \mid \Gamma, \langle \tau \rangle$$

Robotic Arms - Adapted Syntax

```
let (partA , partB) = disassemble product in
```

```
let partB' = clean partB in
```

```
delay  $\tau_{evp}$  ;  
let partB'' = primer (unbox partB') in
```

```
delay  $\tau_{primer\_dry}$  ;  
let partB''' = paint (unbox partB'') in
```

```
delay  $\tau_{paint\_dry}$  ;  
let product' = assemble (partA , unbox partB''') in
```

...

Robotic Arms - Adapted Syntax

```
let (partA , partB) = disassemble product in
# partA and partB available to use below
let partB' = clean partB in

delay  $\tau_{evp}$ ;
let partB'' = primer (unbox partB') in

delay  $\tau_{primer\_dry}$ ;
let partB''' = paint (unbox partB'') in

delay  $\tau_{paint\_dry}$ ;
let product' = assemble (partA , unbox partB''') in

...
```

Robotic Arms - Adapted Syntax

```
let (partA , partB) = disassemble product in
# partA and partB available to use below
let partB' = clean partB in
# cleaning liquid has to evaporate before using partB' below
delay  $\tau_{evp}$ ;
let partB'' = primer (unbox partB') in

delay  $\tau_{primer\_dry}$ ;
let partB''' = paint (unbox partB'') in

delay  $\tau_{paint\_dry}$ ;
let product' = assemble (partA , unbox partB''') in

...
```

Robotic Arms - Adapted Syntax

```
let (partA , partB) = disassemble product in
# partA and partB available to use below
let partB' = clean partB in
# cleaning liquid has to evaporate before using partB' below
delay  $\tau_{evp}$ ;
let partB'' = primer (unbox partB') in
# primer has to dry before using partB'' below
delay  $\tau_{primer\_dry}$ ;
let partB''' = paint (unbox partB'') in

delay  $\tau_{paint\_dry}$ ;
let product' = assemble (partA , unbox partB''') in

...
```

Robotic Arms - Adapted Syntax

```
let (partA , partB) = disassemble product in
# partA and partB available to use below
let partB' = clean partB in
# cleaning liquid has to evaporate before using partB' below
delay  $\tau_{evp}$ ;
let partB'' = primer (unbox partB') in
# primer has to dry before using partB'' below
delay  $\tau_{primer\_dry}$ ;
let partB''' = paint (unbox partB'') in
# paint has to dry before using partB''' below
delay  $\tau_{paint\_dry}$ ;
let product' = assemble (partA , unbox partB''') in

...
```

Robotic Arms - Adapted Syntax

```
let (partA , partB) = disassemble product in
# partA and partB available to use below
let partB' = clean partB in
# cleaning liquid has to evaporate before using partB' below
delay  $\tau_{evp}$ ;
let partB'' = primer (unbox partB') in
# primer has to dry before using partB'' below
delay  $\tau_{primer\_dry}$ ;
let partB''' = paint (unbox partB'') in
# paint has to dry before using partB''' below
delay  $\tau_{paint\_dry}$ ;
let product' = assemble (partA , unbox partB''') in
# product' available to use below
...
```

Expressiveness is Limited in $\lambda_{[\tau]}$

Having $\tau \in \mathbb{N}$:

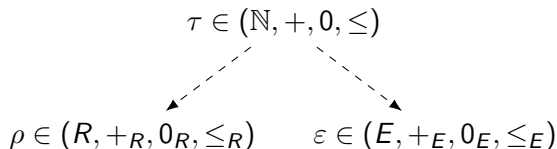
- One cannot indicate until when resources can be used.
- Grades for resources and effects belong to the same structure, $(\mathbb{N}, +, 0, \leq)$.
But the relationship between effects and resources does not imply this is necessarily the case.

Goal of this work

Generalise these grades to other *(ordered) monoids*.

- 1 Motivating Example
- 2 Modal Logic and Modal Types
- 3 The $\lambda_{[\tau]}$ -Calculus
- 4 Generalising $\lambda_{[\tau]}$ – Types
- 5 Generalising $\lambda_{[\tau]}$ – Operational Semantics
- 6 Conclusions and Ongoing/Future Work

Generalising Computations and Types



Adapted grammar for computations, types, and typing contexts:

$M ::= \dots \mid \text{delay } \tau \ M \mid \text{box}_{[\rho]_X} V \text{ as } x \text{ in } M \mid \text{unbox}_{[\rho]_X} V \text{ as } x \text{ in } M$

$EA, EB ::= \text{eb} \mid \text{unit} \mid EA \times EB$

$A, B ::= b \mid \text{unit} \mid A \times B \mid [\rho]_A$

$EX, EY ::= EA \mid EX \times EY$

$X, Y ::= A \mid X \times Y \mid X \rightarrow Y! \varepsilon \mid [\rho]_X$

$C ::= X! \varepsilon$

$\Gamma ::= \emptyset \mid \Gamma, x : X \mid \Gamma, \langle \rho \rangle$

$ -$	:	$E \rightarrow R$
$ -^E$	:	$\mathbb{N} \rightarrow E$
$ -^R$	:	$\mathbb{N} \rightarrow R$

Grades Examples: Intervals of Natural Numbers

	Left-Sided $[a, +\infty)$	Right-Sided $[0, a]$	Two-Sided $[a, b]$
$l_1 \leq_G l_2$	$l_1 := [a_1, +\infty)$ $l_2 := [a_2, +\infty)$ $a_2 \leq a_1$	$l_1 := [0, b_1]$ $l_2 := [0, b_2]$ $b_1 \leq b_2$	$l_1 := [a_1, b_1]$ $l_2 := [a_2, b_2]$ $a_2 \leq a_1$ and $b_1 \leq b_2$
Examples	$[9, +\infty) \leq_G [5, +\infty)$	$[0, 5] \leq_G [0, 8]$	$[5, 8] \leq_G [3, 12]$

Typing Rules - Variable

$$\frac{(x : X) \in \Gamma}{\Gamma \vdash x : X}$$

Typing Rules - Variable

$$\frac{\rho \leq_R 0_R \quad (x : X) \in_\rho \Gamma}{\Gamma \vdash x : X}$$

$$\frac{\rho \leq_R 0_R \quad (x : X) \in_\rho \Gamma}{\Gamma \vdash x : X}$$

Examples:

■ Two-Sided Intervals:

$$\frac{[0, 0] \leq_R [0, 0] \quad (x : \text{float}) \in_{[0,0]} y : Y, \langle [3, 5] \rangle, x : \text{float}}{y : Y, \langle [3, 5] \rangle, x : \text{float} \vdash x : \text{float}}$$

$$\frac{\rho \leq_R 0_R \quad (x : X) \in_\rho \Gamma}{\Gamma \vdash x : X}$$

Examples:

■ Two-Sided Intervals:

$$\frac{[0, 0] \leq_R [0, 0] \quad (x : \text{float}) \in_{[0,0]} y : Y, \langle [3, 5] \rangle, x : \text{float}}{y : Y, \langle [3, 5] \rangle, x : \text{float} \vdash x : \text{float}}$$

■ Left-Sided Intervals:

$$\frac{[3, +\infty) \leq_R [0, +\infty) \quad (x : \text{float}) \in_{[3,+\infty)} y : Y, x : \text{float}, \langle [3, +\infty) \rangle}{y : Y, x : \text{float}, \langle [3, +\infty) \rangle \vdash x : \text{float}}$$

Typing Rules - Variable

$$\frac{\rho \leq_R 0_R \quad (x : X) \in_\rho \Gamma}{\Gamma \vdash x : X}$$

Examples:

■ Two-Sided Intervals:

$$\frac{[3, 5] \leq_R [0, 0] \text{ } \color{red}{\times} \quad \dots}{y : Y, x : \text{float}, \langle [3, 5] \rangle \vdash x : \text{float}}$$

Typing Rules - Variable

$$\frac{(\neg \text{Eternal } X \implies \rho \leq_R 0_R) \quad (x : X) \in_\rho \Gamma}{\Gamma \vdash x : X}$$

Examples:

■ Two-Sided Intervals:

$$\frac{(x : \text{float}) \in_{[3,5]} y : Y, x : \text{float}, \langle [3, 5] \rangle}{y : Y, x : \text{float}, \langle [3, 5] \rangle \vdash x : \text{float}}$$

$$\frac{(\neg \text{Eternal } X \implies \rho \leq_R 0_R) \quad (x : X) \in_\rho \Gamma}{\Gamma \vdash x : X}$$

Examples:

■ Two-Sided Intervals:

$$\frac{(x : \text{float}) \in_{[3,5]} y : Y, x : \text{float}, \langle [3, 5] \rangle}{y : Y, x : \text{float}, \langle [3, 5] \rangle \vdash x : \text{float}}$$

■ Left-Sided Intervals:

$$\frac{[3, +\infty) \leq_R [0, +\infty) \quad (x : \text{float}) \in_{[3, +\infty)} y : Y, x : [[7, +\infty)]\text{float}, \langle [3, +\infty) \rangle}{y : Y, x : [[7, +\infty)]\text{float}, \langle [3, +\infty) \rangle \vdash x : [[7, +\infty)]\text{float}}$$

Typing Rules - Return

$$\underbrace{\Gamma \vdash V : X}_{\text{value typing judgment}} \overline{\Gamma \vdash \text{return } V : X ! 0_E}$$

computation typing judgement

Typing Rules - Sequential Composition

$$\frac{\Gamma \vdash M : Y! \varepsilon \quad \Gamma, \langle \underline{\varepsilon} \rangle, x : Y \vdash N : X! \varepsilon'}{\Gamma \vdash \text{let } x = M \text{ in } N : X! (\underbrace{\varepsilon +_E \varepsilon'}_{\text{total execution time}})}$$

Typing Rules - Sequential Composition

$$\frac{\Gamma \vdash M : Y ! \varepsilon \quad \Gamma, \langle \underline{\varepsilon} \rangle, x : Y \vdash N : X ! \varepsilon'}{\Gamma \vdash \text{let } x = M \text{ in } N : X ! (\underbrace{\varepsilon +_E \varepsilon'}_{\text{total execution time}})}$$

Example:

$$\frac{\frac{\Gamma \vdash \text{clean} : P \rightarrow [\rho_{evp}]CP ! \varepsilon_{clean} \quad \Gamma \vdash \text{product} : P}{\Gamma \vdash \text{clean product} : [\rho_{evp}]CP ! \varepsilon_{clean}} \quad \frac{\Gamma' \vdash \text{product}' : [\rho_{evp}]CP}{\Gamma' \vdash \text{return product}' : [\rho_{evp}]CP ! 0_E}}{\Gamma \vdash \text{let product}' = \text{clean product} \text{ in return product}' : [\rho_{evp}]CP ! (\varepsilon_{clean} +_E 0_E)}$$

where $\Gamma := \text{clean} : P \rightarrow [\rho_{evp}]CP ! \varepsilon_{clean}, \text{product} : P$ and $\Gamma' := \Gamma, \langle \underline{\varepsilon_{clean}} \rangle, \text{product}' : [\rho_{evp}]CP$.

Typing Rules - (Performing) Algebraic Operations

$$\begin{array}{c}
 \text{execution time of op} \\
 \downarrow \\
 \text{input type of op} \qquad \qquad \text{output type of op} \\
 \Gamma \vdash V : X_{\text{op}} \qquad \Gamma, \langle \underline{\varepsilon_{\text{op}}} \rangle, x : Y_{\text{op}} \vdash M : X! \varepsilon \\
 \hline
 \Gamma \vdash \text{op } V (x.M) : X! \varepsilon_{\text{op}} +_E \varepsilon
 \end{array}$$

Typing Rules - (Performing) Algebraic Operations

$$\begin{array}{c}
 \text{execution time of op} \\
 \downarrow \\
 \text{input type of op} \qquad \qquad \text{output type of op} \\
 \Gamma \vdash V : X_{\text{op}} \qquad \Gamma, \langle \underline{\varepsilon_{\text{op}}} \rangle, x : Y_{\text{op}} \vdash M : X! \varepsilon \\
 \hline
 \Gamma \vdash \text{op } V (x.M) : X! \varepsilon_{\text{op}} +_E \varepsilon
 \end{array}$$

Example:

$$\frac{\Gamma \vdash \text{product} : P \quad \frac{\Gamma, \langle \varepsilon_{\text{clean}} \rangle, c : [\rho_{\text{evp}}]CP \vdash c : [\rho_{\text{evp}}]CP}{\Gamma, \langle \varepsilon_{\text{clean}} \rangle, c : [\rho_{\text{evp}}]CP \vdash \text{return } c : [\rho_{\text{evp}}]CP ! 0_E}}{\Gamma \vdash \text{clean product } (c.\text{return } c) : [\rho_{\text{evp}}]CP ! \varepsilon_{\text{clean}}}$$

where **clean** : $P \rightarrow [\rho_{\text{evp}}]CP \vdash \varepsilon_{\text{clean}}$, and $\Gamma := \text{product} : P$.

Typing Rules - Effect Handling

$$\frac{
 \begin{array}{l}
 H = (x.k.M_{\text{op}})_{\text{op} \in \mathcal{O}} \quad \Gamma \vdash M : X! \varepsilon \quad \Gamma, \langle \underline{\varepsilon} \rangle, y : X \vdash N : X! \varepsilon' \\
 \left(\forall \varepsilon''. \Gamma, x : X_{\text{op}}, k : [\underline{\varepsilon_{\text{op}}}] (Y_{\text{op}} \rightarrow X! \varepsilon'') \vdash M_{\text{op}} : X! \varepsilon_{\text{op}} +_E \varepsilon'' \right)_{\text{op} \in \mathcal{O}}
 \end{array}
 }{
 \Gamma \vdash \text{handle } M \text{ with } H \text{ to } y \text{ in } N : X! \varepsilon +_E \varepsilon'
 }$$

Typing Rules - Delay

$$\frac{\Gamma, \langle \boxed{\tau} \rangle \vdash M : A! \varepsilon}{\Gamma \vdash \text{delay } \tau \ M : A! (\boxed{\bar{\tau}} +_E \varepsilon)}$$

Typing Rules - Delay

$$\frac{\Gamma, \langle \underline{\tau} \rangle \vdash M : A ! \varepsilon}{\Gamma \vdash \text{delay } \tau M : A ! (\overline{\tau} +_E \varepsilon)}$$

Example:

$$\frac{\frac{\Gamma, \langle \underline{10} \rangle \vdash \mathbf{dissassemble} : P \rightarrow EX ! \varepsilon_{dasm} \quad \Gamma, \langle \underline{10} \rangle \vdash \mathbf{product} : P}{\Gamma, \langle \underline{10} \rangle \vdash \mathbf{disassemble} \mathbf{product} : EX ! \varepsilon_{dasm}}}{\Gamma \vdash \text{delay } 10 (\mathbf{disassemble} \mathbf{product}) : EX ! (\overline{10} +_E \varepsilon_{dasm})}$$

where $\Gamma := \mathbf{disassemble} : P \rightarrow EX ! \varepsilon_{dasm}, \mathbf{product} : P$.

Typing Rules - Boxing a Resource

$$\frac{\Gamma, \langle \rho \rangle \vdash V : X \quad \Gamma, x : [\rho]X \vdash M : Y ! \varepsilon}{\Gamma \vdash \text{box}_{[\rho]X} V \text{ as } x \text{ in } M : Y ! \varepsilon}$$

Typing Rules - Boxing a Resource

$$\frac{\Gamma, \langle \rho \rangle \vdash V : X \quad \Gamma, x : [\rho]X \vdash M : Y ! \varepsilon}{\Gamma \vdash \text{box}_{[\rho]X} V \text{ as } x \text{ in } M : Y ! \varepsilon}$$

Example: mock the cleaning operation of partB:

$$\frac{\frac{\Gamma, \langle \rho_{\text{evp}} \rangle \vdash \text{newlycpartB} : CP}{\Gamma \vdash \text{box}_{[\rho_{\text{evp}}]CP} \text{newlycpartB} \text{ as } \text{cpartB} \text{ in } \text{return } \text{cpartB} : [\rho_{\text{evp}}]CP ! 0_E} \quad \frac{\Gamma, \text{cpartB} : [\rho_{\text{evp}}]CP \vdash \text{return } \text{cpartB} : [\rho_{\text{evp}}]CP ! 0_E}{\Gamma \vdash \text{box}_{[\rho_{\text{evp}}]CP} \text{newlycpartB} \text{ as } \text{cpartB} \text{ in } \text{return } \text{cpartB} : [\rho_{\text{evp}}]CP ! 0_E}}$$

where $\Gamma := \text{newlycpartB} : CP$.

Typing Rules - Unboxing a Resource

$$\frac{(y : [\rho]X) \in_{\rho'} \Gamma \quad \rho' \leq_R \rho \quad \Gamma, x : X \vdash M : Y ! \varepsilon}{\Gamma \vdash \text{unbox}_{[\rho]X} V \text{ as } x \text{ in } M : Y ! \varepsilon}$$

Typing Rules - Unboxing a Resource

$$\frac{(y : [\rho]X) \in_{\rho'} \Gamma \quad \rho' \leq_R \rho \quad \Gamma, x : X \vdash M : Y ! \varepsilon}{\Gamma \vdash \text{unbox}_{[\rho]X} V \text{ as } x \text{ in } M : Y ! \varepsilon}$$

Example:

$$\frac{\begin{array}{c} partB''' : [\rho_{\text{paint_dry}}]PPB \in_{\rho_{\text{paint_dry}}} \Gamma \\ \vdots \\ \rho_{\text{paint_dry}} \leq_R \rho_{\text{paint_dry}} \end{array} \quad \frac{\frac{\Gamma, upartB''' : PPB \vdash \text{assemble} : (PPA \times PPB) \rightarrow P ! \varepsilon_{asm}}{\Gamma, upartB''' : PPB \vdash partA : PA} \quad \frac{\Gamma, upartB''' : PPB \vdash upartB''' : PPB}{\Gamma, upartB''' : PPB \vdash (partA, upartB''') : P}}{\Gamma, upartB''' : PPB \vdash \text{assemble}(partA, upartB''') : P ! \varepsilon_{asm}} \\ \Gamma \vdash \text{unbox}_{[\rho_{\text{paint_dry}}]PPB} partB''' \text{ as } upartB''' \text{ in } \text{assemble}(partA, upartB''') : [\rho_{\text{clean_evp}}]P ! \varepsilon_{asm}$$

where $\Gamma := \text{assemble} : (PPA \times PPB) \rightarrow P ! \varepsilon_{asm}, partB''' : [\rho_{\text{paint_dry}}]PPB, partA : PA, \langle \rho_{\text{paint_dry}} \rangle$.

Implicit Typing Rule for Sub-Effecting

The following rule is assumed valid implicitly in our type system:

$$\frac{\Gamma \vdash M : X!_{\varepsilon} \quad \boxed{\varepsilon \leq_E \varepsilon'}}{\Gamma \vdash \text{coerce } M : X!_{\varepsilon'}}$$

Implicit Typing Rule for Sub-Effecting

The following rule is assumed valid implicitly in our type system:

$$\frac{\Gamma \vdash M : X ! \varepsilon \quad \varepsilon \leq_E \varepsilon'}{\Gamma \vdash \text{coerce } M : X ! \varepsilon'}$$

The grade ε represents the execution time of the computation.

Interval	Left-Sided $[a, +\infty)$	Right-Sided $[0, a]$	Two-Sided $[a, b]$
$l_1 \leq_G l_2$	$l_1 := [a_1, +\infty)$ $l_2 := [a_2, +\infty)$ $a_2 \leq a_1$	$l_1 := [0, b_1]$ $l_2 := [0, b_2]$ $b_1 \leq b_2$	$l_1 := [a_1, b_1]$ $l_2 := [a_2, b_2]$ $a_2 \leq a_1$ and $b_1 \leq b_2$
Examples	$[9, +\infty) \leq_G [5, +\infty)$	$[0, 5] \leq_G [0, 8]$	$[5, 8] \leq_G [3, 12]$

- Grades can now be units of time, time frames, traces of operations, etc.

Effect and resource grades do not necessarily coincide:

$$x : X, \langle [11, +\infty) \rangle \vdash \text{op } x (y.\text{return } y) : Y ! [1, 5] +_E [0, 0]$$

- Two typing judgements:

$$\Gamma \vdash V : X$$

$$\Gamma \vdash M : X !_\varepsilon$$

- 1 Motivating Example
- 2 Modal Logic and Modal Types
- 3 The $\lambda_{[\tau]}$ -Calculus
- 4 Generalising $\lambda_{[\tau]}$ – Types
- 5 Generalising $\lambda_{[\tau]}$ – Operational Semantics
- 6 Conclusions and Ongoing/Future Work

$$\langle S | M \rangle \rightsquigarrow \langle S' | M' \rangle$$

Where S is a state, that can be either

- $\emptyset$: the empty state
- $S, \langle \rho \rangle$: an extension of S with $\langle \rho \rangle$, indicating that ρ units of time has passed
- $S, x \mapsto_{[\rho]} x \ V$: an extension of S with $x \mapsto_{[\rho]} x \ V$, indicating that V has been boxed up in a computation

$$\langle \mathbb{S} | \text{delay } \tau \ M \rangle \rightsquigarrow \langle \mathbb{S}, \langle \underline{\tau} \rangle | M \rangle$$

$$\langle \mathbb{S} | \text{box}_{[\rho]} x \ V \text{ as } x \text{ in } M \rangle \rightsquigarrow \langle \mathbb{S}, x \mapsto_{[\rho]} V | M \rangle$$

$$\langle \mathbb{S} | \text{unbox}_{[\rho]} x \ V \text{ as } x \text{ in } M \rangle \rightsquigarrow \langle \mathbb{S} | M \{ x \rightarrow \mathbb{S}[V] \} \rangle$$

$$\langle \mathbb{S} | \text{delay } \tau \ M \rangle \rightsquigarrow \langle \mathbb{S}, \langle \underline{\tau} \rangle | M \rangle$$

$$\langle \mathbb{S} | \text{box}_{[\rho]} x \ V \text{ as } x \text{ in } M \rangle \rightsquigarrow \langle \mathbb{S}, x \mapsto_{[\rho]} x \ V | M \rangle$$

$$\langle \mathbb{S} | \text{unbox}_{[\rho]} x \ V \text{ as } x \text{ in } M \rangle \rightsquigarrow \langle \mathbb{S} | M \{ x \rightarrow \mathbb{S}[V] \} \rangle$$

Example on the prototype.

Type Safety Results

Theorem (Preservation)

If $\Gamma_{\mathbb{S}} \vdash M : X! \varepsilon$ and $\langle \mathbb{S} | M \rangle \rightsquigarrow \langle \mathbb{S}' | M' \rangle$, for some $\mathbb{S}'$ and M' , then there exists ε' such that $(\rho_{\mathbb{S}'} +_R \underline{\varepsilon}') \leq_R (\rho_{\mathbb{S}} +_R \underline{\varepsilon})$.

Theorem (Progress)

If $\Gamma_{\mathbb{S}} \vdash M : X! \varepsilon$, then either M is either

- *in a result form,*
- *an algebraic operation call,*
- *or it is possible to make a reduction step from $\langle \mathbb{S} | M \rangle$.*

Type Safety Results

Theorem (Preservation)

If $\Gamma_{\mathbb{S}} \vdash M : X! \varepsilon$ and $\langle \mathbb{S} | M \rangle \rightsquigarrow \langle \mathbb{S}' | M' \rangle$, for some $\mathbb{S}'$ and M' , then there exists ε' such that $(\rho_{\mathbb{S}'} +_R \underline{\varepsilon}') \leq_R (\rho_{\mathbb{S}} +_R \underline{\varepsilon})$.

Theorem (Progress)

If $\Gamma_{\mathbb{S}} \vdash M : X! \varepsilon$, then either M is either

- *in a result form,*
- *an algebraic operation call,*
- *or it is possible to make a reduction step from $\langle \mathbb{S} | M \rangle$.*

Problem: the set of rules do not reflect that operations can alter the state.

(Pure) Top-Level Operational Semantics

$$\frac{\langle S|M \rangle \rightsquigarrow \langle S'|M' \rangle}{\langle S|M \rangle \longmapsto \langle S'|M' \rangle}$$

$$\overline{\langle S|\text{op } V \ (x.M) \rangle \longmapsto \langle S, \langle \varepsilon_{\text{op}} \rangle | M\{x \rightarrow V\} \rangle}$$

(Pure) Top-Level Operational Semantics

$$\frac{\langle \mathbb{S} | M \rangle \rightsquigarrow \langle \mathbb{S}' | M' \rangle}{\langle \mathbb{S} | M \rangle \longmapsto \langle \mathbb{S}' | M' \rangle} \qquad \frac{}{\langle \mathbb{S} |_{\text{op}} V (x.M) \rangle \longmapsto \langle \mathbb{S}, \langle \varepsilon_{\text{op}} \rangle | M \{x \rightarrow V\} \rangle}$$

Theorem (Preservation)

If $\Gamma_{\mathbb{S}} \vdash M : X ! \varepsilon$ and $\langle \mathbb{S} | M \rangle \longmapsto \langle \mathbb{S}' | M' \rangle$, for some $\mathbb{S}'$ and M' , then there exists ε' such that $(\rho_{\mathbb{S}'} +_R \underline{\varepsilon}') \leq_R (\rho_{\mathbb{S}} +_R \underline{\varepsilon})$.

Theorem (Progress)

If $\Gamma_{\mathbb{S}} \vdash M : X ! \varepsilon$, then either M is either

- *in a result form,*
- *or it is possible to make a $(\longmapsto)$ -reduction step from $\langle \mathbb{S} | M \rangle$.*

(Monadic) Top-Level Operational Semantics

So far we stated how the algebraic operation calls affect states,
but the operational semantics still does not reflect the effect of these calls.

(Monadic) Top-Level Operational Semantics

So far we stated how the algebraic operation calls affect states, but the operational semantics still does not reflect the effect of these calls.

$$\frac{\langle S|M \rangle \rightsquigarrow \langle S'|M' \rangle}{\langle S|M \rangle \mapsto_m \text{pure}(\langle S'|M' \rangle)}$$

$$\frac{}{\langle S|\text{op } V \ (x.M) \rangle \mapsto \text{op } V \ \>>= \ \lambda V' \rightarrow \text{pure}(\langle S, \langle \varepsilon_{\text{op}} \rangle | M\{x \rightarrow V'\} \rangle)}$$

(Monadic) Top-Level Operational Semantics

So far we stated how the algebraic operation calls affect states, but the operational semantics still does not reflect the effect of these calls.

$$\frac{\langle \mathbb{S} | M \rangle \rightsquigarrow \langle \mathbb{S}' | M' \rangle}{\langle \mathbb{S} | M \rangle \mapsto_m \text{pure}(\langle \mathbb{S}' | M' \rangle)}$$
$$\frac{}{\langle \mathbb{S} | \text{op } V (x.M) \rangle \mapsto \text{op } V \text{ } \gg \text{ } \lambda V' \rightarrow \text{pure}(\langle \mathbb{S}, \langle \varepsilon_{\text{op}} \rangle | M\{x \rightarrow V'\}\rangle)}$$

The Preservation and Progress theorems still hold.

- 1 Motivating Example
- 2 Modal Logic and Modal Types
- 3 The $\lambda_{[\tau]}$ -Calculus
- 4 Generalising $\lambda_{[\tau]}$ – Types
- 5 Generalising $\lambda_{[\tau]}$ – Operational Semantics
- 6 Conclusions and Ongoing/Future Work

Conclusions:

- Graded resources and graded effect systems are useful to manage time-sensitive resources.
- We extended the framework of Ahman and Žajdela so that now:
 - Resources and effects can be modelled with different grades.
 - Our system captures different time-sensitivity (e.g., expiring time, upper or lower time-bonds, etc).
 - The stateful operational semantics reflects the effects algebraic operation calls have on computations.

Ongoing work:

- Capture concurrent/parallel settings.
- Provide a type-inference algorithm.
- Equational theory: prove the equational soundness result.



Thanks for your
attention!

Type System Rules

$$\begin{array}{c}
 \frac{(\neg \text{Eternal } X \implies \rho \leq_R 0_R) \quad (x : X) \in_\rho \Gamma}{\Gamma \vdash x : X} \quad \frac{(\Gamma \vdash V_i : b_i)_{1 \leq i \leq n}}{\Gamma \vdash f(V_1, \dots, V_n) : b} \quad \frac{\Gamma \vdash V : X \quad \Gamma \vdash W : Y}{\Gamma \vdash (V, W) : X \times Y} \\
 \\
 \frac{}{\Gamma \vdash () : \text{unit}} \quad \frac{\Gamma, x : X \vdash M : Y ! \varepsilon}{\Gamma \vdash \text{fun } (x : X) \mapsto M : X \rightarrow Y ! \varepsilon} \quad \frac{\Gamma \vdash V : X}{\Gamma \vdash \text{return } V : X ! 0_E} \\
 \\
 \frac{\Gamma \vdash M : X ! \varepsilon \quad \Gamma, \langle \underline{\varepsilon} \rangle, x : X \vdash N : Y ! \varepsilon'}{\Gamma \vdash \text{let } x = M \text{ in } N : Y ! \varepsilon +_E \varepsilon'} \quad \frac{\Gamma \vdash V : X \rightarrow Y ! \varepsilon \quad \Gamma \vdash W : X}{\Gamma \vdash V W : Y ! \varepsilon} \\
 \\
 \frac{\Gamma \vdash V : X \times Y \quad \Gamma, x : X, y : Y \vdash M : Z ! \varepsilon}{\Gamma \vdash \text{match } V \text{ with } \{(x, y) \mapsto y\} : Z ! \varepsilon} \quad \frac{\Gamma \vdash V : X_{\text{op}} \quad \Gamma, \langle \underline{\varepsilon}_{\text{op}} \rangle, x : Y_{\text{op}} \vdash M : X ! \varepsilon}{\Gamma \vdash \text{op } V (x.M) : X ! \varepsilon_{\text{op}} +_E \varepsilon} \\
 \\
 \frac{\Gamma, \langle \underline{\tau} \rangle \vdash M : X ! \varepsilon}{\Gamma \vdash \text{delay } \tau M : X ! \bar{\tau} +_E \varepsilon} \\
 \\
 \frac{\Gamma, \langle \varepsilon \rangle, y : X \vdash N : X ! \varepsilon' \quad \Gamma \vdash V : X_{\text{op}} \quad H = (x.k.M_{\text{op}})_{\text{op} \in \mathcal{O}} \quad \Gamma \vdash M : X ! \varepsilon \quad \left(\forall \varepsilon'' . \Gamma, x : X_{\text{op}}, k : [\underline{\varepsilon}_{\text{op}}](Y_{\text{op}} \rightarrow X ! \varepsilon'') \vdash M_{\text{op}} : X ! \varepsilon_{\text{op}} +_E \varepsilon'' \right)_{\text{op} \in \mathcal{O}}}{\Gamma \vdash \text{handle } M \text{ with } H \text{ to } y \text{ in } N : X ! \varepsilon +_E \varepsilon'} \\
 \\
 \frac{\Gamma, \langle \rho \rangle \vdash V : X \quad \Gamma, x : [\rho]X \vdash M : Y ! \varepsilon}{\Gamma \vdash \text{box}_{[\rho]X} V \text{ as } x \text{ in } M : Y ! \varepsilon} \quad \frac{(y : [\rho]X) \in_{\rho'} \Gamma \quad \rho' \leq_R \rho \quad \Gamma, x : X \vdash M : Y ! \varepsilon}{\Gamma \vdash \text{unbox}_{[\rho]X} V \text{ as } x \text{ in } M : Y ! \varepsilon}
 \end{array}$$

Reduction Rules

$$\frac{}{\langle \mathbb{S} | (\text{fun}(x : X) \mapsto M) V \rangle \rightsquigarrow \langle \mathbb{S} | M\{x \rightarrow V\} \rangle}$$

$$\frac{}{\langle \mathbb{S} | \text{match } V \text{ with } \{(W, x) \mapsto x\} M \rangle \rightsquigarrow \langle \mathbb{S} | M\{x \rightarrow V\} \{y \rightarrow W\} \rangle}$$

$$\langle \mathbb{S} | M \rangle \rightsquigarrow \langle \mathbb{S}' | M' \rangle$$

$$\frac{}{\langle \mathbb{S} | \text{let } x = M \text{ in } N \rangle \rightsquigarrow \langle \mathbb{S}' | \text{let } x = M' \text{ in } N \rangle}$$

$$\frac{}{\langle \mathbb{S} | \text{let } x = \text{return } V \text{ in } M \rangle \rightsquigarrow \langle \mathbb{S} | M\{x \rightarrow V\} \rangle}$$

$$\frac{}{\langle \mathbb{S} | \text{let } x = \text{op } V (y.M) \text{ in } N \rangle \rightsquigarrow \langle \mathbb{S} | \text{op } V (y.\text{let } x = M \text{ in } N) \rangle}$$

$$\frac{}{\langle \mathbb{S} | \text{let } x = \text{op } V (y.M) \text{ in } N \rangle \rightsquigarrow \langle \mathbb{S} | \text{op } V (y.\text{let } x = M \text{ in } N) \rangle}$$

$$\frac{}{\langle \mathbb{S} | \text{delay } \tau M \rangle \rightsquigarrow \langle \mathbb{S}, \langle \tau \rangle | M \rangle}$$

$$\frac{}{\langle \mathbb{S} | \text{handle } (\text{return } V) \text{ with } H \text{ to } x \text{ in } M \rangle \rightsquigarrow \langle \mathbb{S} | M\{x \rightarrow V\} \rangle}$$

$$\langle \mathbb{S} | M \rangle \rightsquigarrow \langle \mathbb{S}' | M' \rangle$$

$$\frac{}{\langle \mathbb{S} | \text{handle } M \text{ with } H \text{ to } x \text{ in } N \rangle \rightsquigarrow \langle \mathbb{S}' | \text{handle } M' \text{ with } H \text{ to } x \text{ in } N \rangle}$$

$$H = (x.k.M_{\text{op}})$$

$$\frac{}{\langle \mathbb{S} | \text{handle } (\text{op } V (y.M)) \text{ with } H \text{ to } x \text{ in } M \rangle \rightsquigarrow \langle \mathbb{S}' | \text{box}_{\text{f}} \text{ fun}(x:\text{as}) \mapsto y \text{ in } Y_{\text{op}} \text{handle } M \text{ with } H \text{ to } z \text{ in } M_{\text{op}}\{x \rightarrow V\} \rangle}$$

$$\frac{}{\langle \mathbb{S} | \text{box}_{[\rho]X} V \text{ as } x \text{ in } M \rangle \rightsquigarrow \langle \mathbb{S}, x \mapsto_{[\rho]X} V | M \rangle}$$

$$\frac{}{\langle \mathbb{S} | \text{unbox}_{[\rho]X} V \text{ as } x \text{ in } M \rangle \rightsquigarrow \langle \mathbb{S} | M\{x \rightarrow \mathbb{S}[V]\} \rangle}$$

$$\frac{}{\langle \mathbb{S} | \text{coerce } M \rangle \rightsquigarrow \langle \mathbb{S} | M \rangle}$$